

Week-23 | Azure Zero to Hero | Getting Started with Terraform for Azure



Introduction

Terraform is an Infrastructure as Code (IaC) tool that allows the provisioning, management, and automation of infrastructure resources in cloud environments. It supports multiple cloud providers such as Azure, AWS, and GCP, providing a unified way to manage resources. This document covers the foundational concepts of using Terraform with Azure, including setting up the environment, configuring resources, and best practices for state management.

Why Use Terraform for Azure?

Azure provides several automation tools like Azure Resource Manager (ARM) templates, Azure CLI, and SDKs for managing infrastructure. However, Terraform offers:

- **Multi-cloud support:** A unified approach to managing infrastructure across multiple cloud providers.
 - **Declarative configuration:** Infrastructure is described in a human-readable format (HCL – HashiCorp Configuration Language).
 - **State management:** Keeps track of infrastructure changes to maintain consistency.
 - **Modularity:** Reusable modules simplify infrastructure deployment.
 - **Version control:** Infrastructure as Code (IaC) can be managed using Git for versioning and collaboration.
-

Setting Up Terraform for Azure

Before using Terraform to manage Azure resources, it is essential to set up the environment.

1. Installing Required Tools

To begin, install the following:

- **Visual Studio Code (VS Code):** The recommended editor for writing and managing Terraform configurations.
- **Azure CLI:** Used to authenticate Terraform with Azure. Install it using:
`curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash`
- **Terraform:** Install Terraform by downloading it from [HashiCorp's official website](https://www.hashicorp.com/products/terraform) or using:
`sudo apt-get update && sudo apt-get install -y terraform`
- **Azure Terraform Extension:** Available in the VS Code marketplace for enhanced Terraform support.

Authenticating Terraform with Azure

To allow Terraform to manage Azure resources, authentication is required.

Login Using Azure CLI

1. Open a terminal and log in:
2. `az login`
3. Select the appropriate subscription if multiple exist:
4. `az account set --subscription "YOUR_SUBSCRIPTION_ID"`
5. Verify authentication by checking the active subscription:
6. `az account show`

Writing Terraform Configuration for Azure

Terraform configurations are written in .tf files using HCL. The configuration consists of:

- **Provider declaration**
- **Resource definitions**
- **State management configurations**

1. Declaring Azure Provider

Create a new directory for the Terraform project and a main.tf file:

```
mkdir terraform-azure
```

```
cd terraform-azure
```

nano main.tf

Add the following provider block in main.tf:

```
provider "azurerm" {  
  features {}  
}
```

2. Creating Azure Resources

Define a **Resource Group** in Terraform:

```
resource "azurerm_resource_group" "my_rg" {  
  name     = "terraform-rg"  
  location = "East US"  
}
```

Define a **Virtual Network and Subnet**:

```
resource "azurerm_virtual_network" "my_vnet" {  
  name                = "terraform-vnet"  
  location             = azurerm_resource_group.my_rg.location  
  resource_group_name = azurerm_resource_group.my_rg.name  
  address_space       = ["10.0.0.0/16"]  
}  
  
resource "azurerm_subnet" "my_subnet" {  
  name                = "terraform-subnet"  
  resource_group_name = azurerm_resource_group.my_rg.name  
  virtual_network_name = azurerm_virtual_network.my_vnet.name  
  address_prefixes     = ["10.0.1.0/24"]  
}
```

Define a **Network Interface and Virtual Machine**:

```
resource "azurerm_network_interface" "my_nic" {  
  name                = "terraform-nic"  
  location             = azurerm_resource_group.my_rg.location
```

```
resource_group_name = azurerm_resource_group.my_rg.name
```

```
ip_configuration {  
  name          = "internal"  
  subnet_id     = azurerm_subnet.my_subnet.id  
  private_ip_address_allocation = "Dynamic"  
}  
}
```

```
resource "azurerm_linux_virtual_machine" "my_vm" {  
  name          = "terraform-vm"  
  resource_group_name = azurerm_resource_group.my_rg.name  
  location      = azurerm_resource_group.my_rg.location  
  size          = "Standard_B1s"  
  admin_username = "azureuser"  
  network_interface_ids = [azurerm_network_interface.my_nic.id]  
  os_disk {  
    caching          = "ReadWrite"  
    storage_account_type = "Standard_LRS"  
  }  
}
```

```
source_image_reference {  
  publisher = "Canonical"  
  offer     = "UbuntuServer"  
  sku       = "18.04-LTS"  
  version   = "latest"  
}
```

```
admin_ssh_key {  
  username = "azureuser"  
  public_key = file("~/ssh/id_rsa.pub")  
}
```

```
}  
  
}
```

Executing Terraform Commands

Once the Terraform configuration is written, execute the following commands:

1. Initialize Terraform

`terraform init`

This downloads the necessary provider plugins and sets up the Terraform environment.

2. Plan Execution

`terraform plan`

This generates an execution plan and shows the resources that will be created.

3. Apply the Configuration

`terraform apply -auto-approve`

Terraform creates the resources in Azure.

4. Verify the Deployment

Check the created resources in the Azure Portal or use:

`az resource list --resource-group terraform-rg --output table`

5. Destroy Resources (Optional)

To delete all resources:

`terraform destroy -auto-approve`

Managing Terraform State

Terraform maintains a **state file** (`terraform.tfstate`) to track resources. By default, this file is stored locally, but in a production environment, storing it in a remote backend such as Azure Blob Storage is recommended.

1. Configure Remote State Storage in Azure

Create an Azure Storage Account and a Blob Container:

`az storage account create --name terraformstate123 --resource-group terraform-rg --location eastus --sku Standard_LRS`

`az storage container create --name terraformstate --account-name terraformstate123`

Retrieve the storage account access key:

```
az storage account keys list --resource-group terraform-rg --account-name terraformstate123
--query "[0].value" --output tsv
```

Configure Terraform backend in main.tf:

```
terraform {
  backend "azurerm" {
    resource_group_name = "terraform-rg"
    storage_account_name = "terraformstate123"
    container_name      = "terraformstate"
    key                 = "terraform.tfstate"
  }
}
```

Reinitialize Terraform to migrate the state file:

```
terraform init
```

Best Practices

1. **Use Variables:** Avoid hardcoding values using variables.tf. Example:

```
variable "location" {
  default = "East US"
}
```
 2. **Use Modules:** Organize infrastructure code using reusable modules.
 3. **Secure Secrets:** Store sensitive data like access keys in **Azure Key Vault**.
 4. **Enable Logging:** Use Terraform logs for debugging with:

```
export TF_LOG=DEBUG
```
 5. **Implement CI/CD:** Automate Terraform execution using GitHub Actions or Azure DevOps.
 6. **Lock State File:** Prevent simultaneous updates by enabling state file locking.
-

Conclusion

Terraform provides an efficient way to automate and manage Azure infrastructure. By using best practices such as remote state storage, modularization, and CI/CD integration, teams can ensure scalable and secure infrastructure deployment. Whether deploying simple virtual machines or complex multi-tier applications, Terraform streamlines infrastructure provisioning in Azure.

